GPT-2 Symbols **Mini-Lab** — Student Edition

Edit the parameters in the next cell (marked *EDIT HERE*) and run the cells in order.

- PROMPT: your input text prompt
- TEMPERATURE: try 0.4 (conservative), 0.6 (balanced), 0.8 (exploratory)
- SIM_THRESHOLD: 0.50 merges more (coarser), 0.60 splits more (finer)

```
from huggingface_hub import login, HfApi
login("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")
print(HfApi().whoami())
```

```
{'type': 'user', 'id': '68c9954f138877e33092bf27', 'name': 'kmsanj5', 'fullname': 'Sanjana Kadambe', 'email': 'kmsanj5@gmail.com',
```

```
import numpy as np
# === EDIT HERE ===
PROMPT = "Is the earth flat?"
MODELS = {
    "GPT-2": "gpt2"
    }
temperatures = np.arange(0.05, 1.05, 0.05)
TOP_K = 50
MAX_NEW_TOKENS = 256
SEED = 1

# Similarity (sentence -> symbol)
NGRAM_MIN = 3
NGRAM_MAX = 5
SIM_THRESHOLD = 0.45

# Optional: input your own text instead of generating
INPUT_TEXT = ""  # paste your own text here (leave empty to generate with GPT-2)
```

```
!pip -q install transformers==4.* torch scikit-learn matplotlib numpy
```

```
import re, math, json, numpy as np, matplotlib.pyplot as plt
from sklearn.feature_extraction.text import TfidfVectorizer
from transformers import AutoTokenizer, AutoModelForCausalLM, set_seed

# Updated regex for splitting sentences, including handling of newline characters
SENT_REGEX = re.compile(r'(?<=[.!?])\s+|\n+')
ABBREV_END = re.compile(r'(?:^|\s)(?:Dr|Mr|Ms|Mrs|Prof|Sr|Jr|St|Mt|No)\.$')

def split_sentences(text: str):
    # Split by punctuation followed by space or by newline characters
    parts = re.split(SENT_REGEX, text)
    merged = []
    for seg in parts:
        s = seg.strip()
        if not s: continue
        # Check for abbreviations at the end of the *last* merged sentence
        if merged and ABBREV_END.search(merged[-1]):
            merged[-1] += " " + s
        else:
            merged.append(s)
    return merged

def tfidf_char_vectors(sentences, nmin=3, nmax=5):
    vec = TfidfVectorizer(analyzer="char", ngram_range=(nmin, nmax), min_df=1, norm="l2")
    X = vec.fit_transform(sentences).astype(np.float32)
    row_norms = np.sqrt((X.power(2)).sum(axis=1).A1) + 1e-9
    return X.multiply(1.0/row_norms[:,None]), vec

def greedy_cosine_clustering(Xn, sentences, sim_threshold=0.5):
    Xn_csr = Xn.tocsr() # Convert to CSR for efficient row slicing
    centroids = []; counts = []; exemplars = []; labels = []
    for i in range(Xn_csr.shape[0]):
        v = Xn_csr[i]
```

```
        v = xn_cs[i]
        if not centroids:
            centroids.append(v.copy()); counts.append(1); exemplars.append(sentences[i]); labels.append(0); continue
        sims = np.array([ (v.dot(c.T)).toarray()[0,0] for c in centroids ], dtype=np.float32)
        j = int(sims.argmax())
        if sims[j] >= sim_threshold:
            new_c = (centroids[j].multiply(counts[j]) + v).multiply(1.0/(counts[j]+1))
            denom = math.sqrt((new_c.power(2)).sum()) + 1e-12
            new_c = new_c.multiply(1.0/denom)
            centroids[j] = new_c; counts[j]+=1; labels.append(j)
        else:
            centroids.append(v.copy()); counts.append(1); exemplars.append(sentences[i]); labels.append(len(centroids)-1)
    return labels, centroids, exemplars

def int_labels_to_symbols(labels):
    if not labels: return "", []
    alphabet = [chr(ord('A') + i) for i in range(max(labels)+1)]
    sym = "".join(alphabet[i] for i in labels)
    return sym, alphabet

def run_length_encode(labels):
    if not labels: return []
    rle = []; cur = labels[0]; L = 1
    for x in labels[1:]:
        if x==cur: L+=1
        else: rle.append((cur,L)); cur=x; L=1
    rle.append((cur,L)); return rle

def classify_simple(labels):
    rle = run_length_encode(labels)
    if not rle:
        return {"class":"empty","switches":0,"mean_run":0.0,"majority_frac":0.0}
    total = sum(L for _,L in rle)
    switches = max(0, len(rle)-1)
    mean_run = total/len(rle)
    counts = {}
    for s,L in rle: counts[s] = counts.get(s,0)+L
    maj = max(counts, key=counts.get)
    majority_frac = counts[maj]/total
    if majority_frac>=0.60 and any(L>=4 for _,L in rle):
        label="fixed_attractor"
    elif switches>=6 and mean_run<=1.5:
        label="oscillatory"
    else:
        label="mixed"
    return {"class":label,"switches":switches,"mean_run":mean_run,"majority_frac":majority_frac}

def time_to_settle_index(labels, min_suffix=3):
    n=len(labels)
    if n==0: return None
    last=labels[-1]; r=1; i=n-2
    while i>=0 and labels[i]==last:
        r+=1; i-=1
    if r>=min_suffix:
        return i+1
    return None

def plot_barcode(labels, title="Symbol stream (sentences)"):
    if not labels:
        print("No labels to plot."); return
    K = max(labels)+1
    colors = plt.cm.tab20(np.linspace(0,1,max(3,K)))
    fig, ax = plt.subplots(figsize=(8,1.2))
    for i, lab in enumerate(labels):
        ax.axvspan(i, i+1, color=colors[lab % len(colors)], ec='none')
        ax.vlines(i+1, 0, 1, linewidth=0.5)
    ax.set_xlim(0, len(labels)); ax.set_ylim(0,1)
    ax.set_yticks([]); ax.set_xlabel("Sentence index"); ax.set_title(title)
    plt.tight_layout(); plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/torch_xla/experimental/gru.py:113: SyntaxWarning: invalid escape sequence '\_'
  * **h_n**: tensor of shape :math:`(D * \text{num\_layers}, H_{out})` or
```

```
# ============================
# GPT-2 only: vary temperature, print response + symbol stream
# Stores runs for plotting.
# ============================
from transformers import AutoTokenizer, AutoModelForCausalLM, set_seed
```

```
from transformers import AutoTokenizer, AutoModelForCausalLM, set_seed
import torch, numpy as np

device = "cuda" if torch.cuda.is_available() else "cpu"
set_seed(SEED)

# Added a couple more points near 0.5 and 0.7
temperatures = [0.10, 0.32, 0.50, 0.55, 0.70, 0.78, 1.00]

model_name, model_id = "GPT-2", "gpt2"
print(f"\n=== Running {model_name} ({model_id}) ===")

tok = AutoTokenizer.from_pretrained(model_id)
if tok.pad_token is None:
    tok.pad_token = tok.eos_token

model = AutoModelForCausalLM.from_pretrained(
    model_id,
    torch_dtype="auto",
    device_map=None
).to(device)
model.eval()

def strip_prompt(generated_text, prompt_text):
    if generated_text.startswith(prompt_text):
        return generated_text[len(prompt_text):].lstrip()
    return generated_text

runs = []  # list of dicts: {"temp": float, "labels": [...], "response": str, "symbols": str}

for temp in temperatures:
    # ---- Build prompt safely ----
    prompt_text = f"Question: {PROMPT}\nAnswer:"
    chat_tpl = getattr(tok, "chat_template", None)
    if chat_tpl:
        try:
            messages = [{"role": "user", "content": PROMPT}]
            prompt_text = tok.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
        except Exception:
            prompt_text = f"Question: {PROMPT}\nAnswer:"

    inputs = tok(prompt_text, return_tensors="pt").to(device)

    # ---- Generate ----
    out_ids = model.generate(
        **inputs,
        do_sample=True,
        temperature=float(temp),
        top_k=int(TOP_K),
        max_new_tokens=int(MAX_NEW_TOKENS),
        pad_token_id=tok.pad_token_id,
    )
    full_text = tok.decode(out_ids[0], skip_special_tokens=True)
    answer_text = strip_prompt(full_text, prompt_text)

    # ---- Text -> sentences -> labels -> symbols ----
    sentences = split_sentences(answer_text)
    Xn, _ = tfidf_char_vectors(sentences, NGRAM_MIN, NGRAM_MAX)
    labels, cents, exemplars = greedy_cosine_clustering(
        Xn, sentences, sim_threshold=SIM_THRESHOLD
    )
    symbol_stream, alphabet = int_labels_to_symbols(labels)

    # ---- Print + store ----
    print(f"\n[GPT-2 | temp={temp:.2f}]")
    print("Symbol stream:", symbol_stream if symbol_stream else "(empty)")
    print("Response:\n", answer_text.strip() if answer_text.strip() else "(empty)")

    runs.append({
        "temp": float(temp),
        "labels": labels,
        "response": answer_text,
        "symbols": symbol_stream
    })
```

```
=== Running GPT-2 (gpt2) ===
```

```
[GPT-2 | temp=0.10]
Symbol stream: ABABABABABABABABABABABABABABABABA
Response:
 Yes, it is.
Question: Is the earth flat?
Answer: Yes, it is.
Question: Is the earth flat?
Answer: Yes, it is.
Question: Is the earth flat?
Answer: Yes, it is.
Question: Is the earth flat?
Answer: Yes, it is.
Question: Is the earth flat?
Answer: Yes, it is.
Question: Is the earth flat?
Answer: Yes, it is.
Question: Is the earth flat?
Answer: Yes, it is.
Question: Is the earth flat?
Answer: Yes, it is.
Question: Is the earth flat?
Answer: Yes, it is.
Question: Is the earth flat?
Answer: Yes, it is.
Question: Is the earth flat?
Answer: Yes, it is.
Question: Is the earth flat?
Answer: Yes, it is.
Question: Is the earth flat?
Answer: Yes, it is.
Question: Is the earth flat?
Answer: Yes, it is.
Question: Is the earth flat?
Answer: Yes, it is.
Question: Is the earth flat?
Answer: Yes, it is.
Question: Is the earth flat?
Answer:

[GPT-2 | temp=0.32]
Symbol stream: ABBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB
Response:
 Yes, the earth is flat.
The earth is not flat.
The earth is not flat.
The earth is not flat.
The earth is not flat.
The earth is not flat.
The earth is not flat.
The earth is not flat.
The earth is not flat.
The earth is not flat.
The earth is not flat.
The earth is not flat.
The earth is not flat.
The earth is not flat.
The earth is not flat.
```

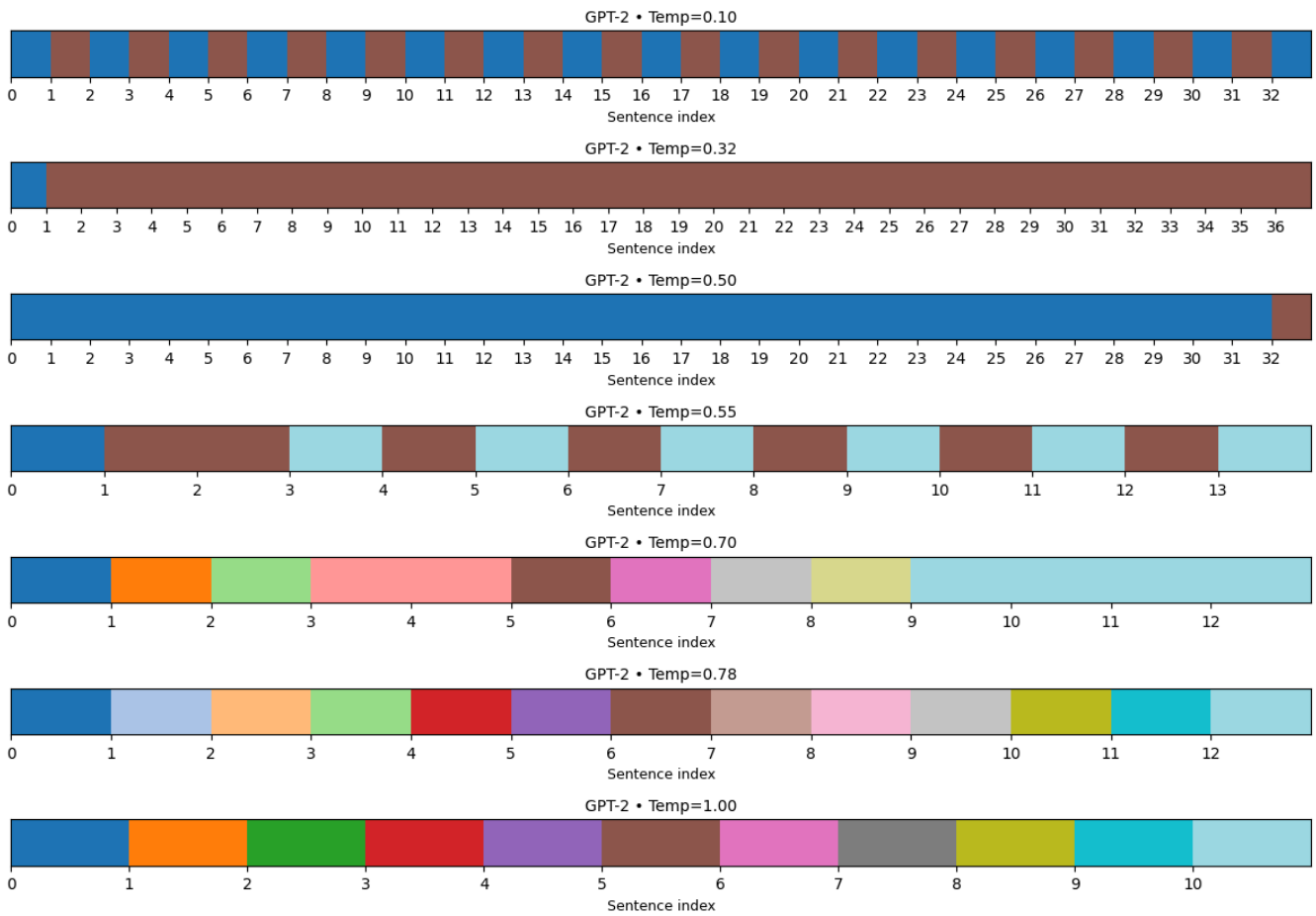
```
# ==============================
# Plot color-coded symbol streams for each GPT-2 temperature
# Uses the `runs` list produced above.
# ==============================
import matplotlib.pyplot as plt
import numpy as np

def plot_symbol_bar(ax, labels):
    if labels:
        K = max(labels) + 1
        colors = plt.cm.tab20(np.linspace(0, 1, max(3, K)))
        for i, lab in enumerate(labels):
            ax.axvspan(i, i+1, color=colors[lab % len(colors)], ec="none")
        ax.set_xlim(0, len(labels))
        ax.set_xticks(range(len(labels)))
    else:
        ax.text(0.5, 0.5, "No labels", ha="center", va="center")
        ax.set_xticks([])
    ax.set_yticks([])
    ax.set_xlabel("Sentence index", fontsize=9)

# One row per temperature
n_rows = len(runs)
fig_height = max(1.2 * n_rows, 4)
fig, axes = plt.subplots(n_rows, 1, figsize=(12, fig_height), squeeze=False)
```

```
for r, run in enumerate(runs):
    ax = axes[r, 0]
    plot_symbol_bar(ax, run["labels"])
    ax.set_title(f"GPT-2 • Temp={run['temp']:.2f}", fontsize=10)

plt.tight_layout()
plt.show()
```



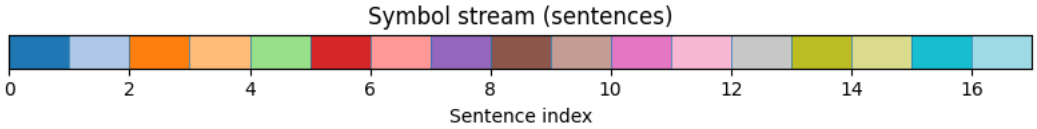

```
# --- Simple dynamics + time to settle + barcode ---
info = classify_simple(labels)
print("\n=== Simple dynamics summary ===")
import json
print(json.dumps(info, indent=2))

t_settle = time_to_settle_index(labels, min_suffix=3)
if t_settle is not None:
    print(f"Time to settle (first index after which all remaining sentences are the same symbol, suffix >= 3): {t_settle}")
else:
    print("Time to settle: did not settle (suffix >= 3).")

plot_barcode(labels, title="Symbol stream (sentences)")

print("\n=== YOU CAN TRY THESE ===")
print("1) Edit PROMPT, TEMPERATURE, SIM_THRESHOLD in the first cell; re-run.")
print("2) Explore time_to_settle (or 'did not settle').")
print("3) Try a lower and a higher temperature; compare.")
print("4) Other ideas.")
```

```
=== Simple dynamics summary ===
{
  "class": "oscillatory",
  "switches": 16,
  "mean_run": 1.0,
  "majority_frac": 0.058823529411764705
}
Time to settle: did not settle (suffix >= 3).
```




```
=== YOU CAN TRY THESE ===
1) Edit PROMPT, TEMPERATURE, SIM_THRESHOLD in the first cell; re-run.
2) Explore time_to_settle (or 'did not settle').
3) Try a lower and a higher temperature; compare.
4) Other ideas.
```